Persian version: ../01-project-overview.html

Organizational Meeting Assistant Project Overview & Vision

Document 1 of 3 · Thesis / capstone project · AI-native product design · Repository: github.com/Rvin-zh/meeting-assistant · CHANGELOG

The **Organizational Meeting Assistant** turns raw meeting conversation — whether typed text or an audio file/recording — into **structured, actionable knowledge**: a clean summary, key points, explicit decisions, follow-up tasks, a question-and-answer layer over meeting content, and one-click Jira issue creation. This product is the first deliverable from an extensible core we call the *LLM-based Organizational AI Core*.

Contents

1. The Problem

In organizations and companies — at any scale — a continuous stream of meetings takes place: formal sessions (board, project steering, client reviews) and informal team conversations (daily standups, ad-hoc coordination). These meetings consume a large share of an organization's most expensive resource — people's attention and time — yet in many cases they produce **no durable, trackable output**.

Failure patterns in this process are well known:

- **Decisions evaporate.** A verbal decision is made, everyone agrees, and a week later no one remembers exactly what was agreed or why.
- **Tasks get lost.** Action items live in personal notes, scatter across chat threads, or are never recorded — ownership and deadlines disappear.
- **Meetings are not searchable.** Months later, when someone asks ("Did we decide to drop feature X?"), there is no way to query past meetings.
- **Tools are fragmented.** Calendar, email, chat, and project management live in separate islands; manually keeping them aligned rarely happens.

The result is wasted time, forgotten commitments, unclear accountability, and decisions that are never followed up. As organizations move toward agility and automation, the absence of reliable "meeting memory" becomes a measurable productivity barrier.

2. Product Vision

One-liner	An extensible organizational AI assistant core, personalized on each organization's data.
Technical name	`LLM-based Organizational AI Core`
Market name	Platform for building custom organizational AI assistants
First product (this MVP)	Meeting Assistant — transcript ← summary · tasks · RAG · Jira

The strategic bet is that most organizations do not want a generic chatbot; they want an assistant that understands *their meetings*, *their projects*, and *their tools*. The Meeting Assistant is the wedge: a high-frequency, high-pain workflow that

proves the core's value and produces the proprietary data (decisions, tasks, org documents) that makes future capabilities defensible.

This means the product is designed as a platform from day one: meeting analysis is only the first *skill* in a set of skills that run on the same LLM core, the same retrieval layer, and the same data infrastructure.

3. What the MVP Does Today

The current, demo-ready product covers a complete end-to-end flow:

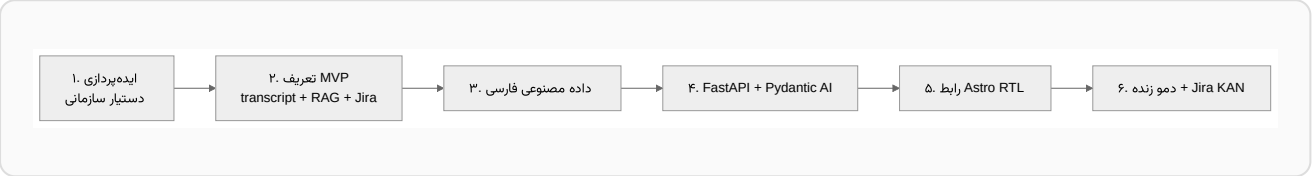
- **Flexible input** — Paste a Persian transcript with speaker labels, upload a text file, or provide **audio** (upload `mp3/wav/m4a/ogg/webm/flac` or record directly with the browser microphone).
- **Audio transcription** — Uploaded or recorded audio is transcribed with Gemini via the same `GOOGLE_API_KEY`, producing an editable transcript before analysis.
- **Automatic analysis** — Extraction of **summary**, **key points**, explicit **decisions**, and structured **tasks** (title, assignee, deadline, priority).
- **Ask the meeting (RAG)** — A natural-language Q&A layer scoped to one meeting, with source attribution.
- **Smart conversational behavior** — Greetings like "hello" are answered directly without searching the transcript; only real questions trigger retrieval.
- **Jira integration** — Preview and create issues in a Jira Cloud project (issues are written in **English** for clean LTR display, while the UI remains Persian).
- **Facilitator guide** — After each meeting, the assistant offers coaching-style suggestions for running the next session better (details in documents 2 and 3).

> **Intentional MVP scope.** This is a demo-ready product, not a finished platform. Live streaming transcription (streaming STT) *during* a meeting, professional speaker diarization, and multi-user authentication are deliberately out of MVP scope and planned in the roadmap (document 3).

4. Academic Framing: An AI-Native Product

Beyond a practical tool, this project is structured as a case study in **AI-native product design** — a journey from idea to a deployable MVP for a product whose core value is *the AI capability itself*, not a conventional application with AI bolted on.

4.1 From Idea to MVP



Each stage reflects a strand of modern AI product engineering: choosing a problem where LLMs have real advantage; defining the smallest feature set that proves value; preparing representative data; selecting a lightweight, typed agent framework; designing a usable interface; and validating against explicit acceptance criteria.

4.2 Why RAG and an Agent Framework

Two ideas from contemporary AI practice anchor the design:

- **Retrieval-Augmented Generation (RAG)** grounds assistant answers in actual meeting content instead of the model's parametric memory, keeping responses faithful and attributable.
- **Typed agent orchestration** (with Pydantic AI) gives every model call a strict output contract, so the system produces validated data structures — not free text that downstream code must guess at.

We deliberately chose a *lightweight* agent framework over a heavy orchestration stack (such as LangChain): the MVP needs typed outputs and native Gemini integration, not a large dependency surface.

4.3 Product Place in the MLOps / AIOps Lifecycle

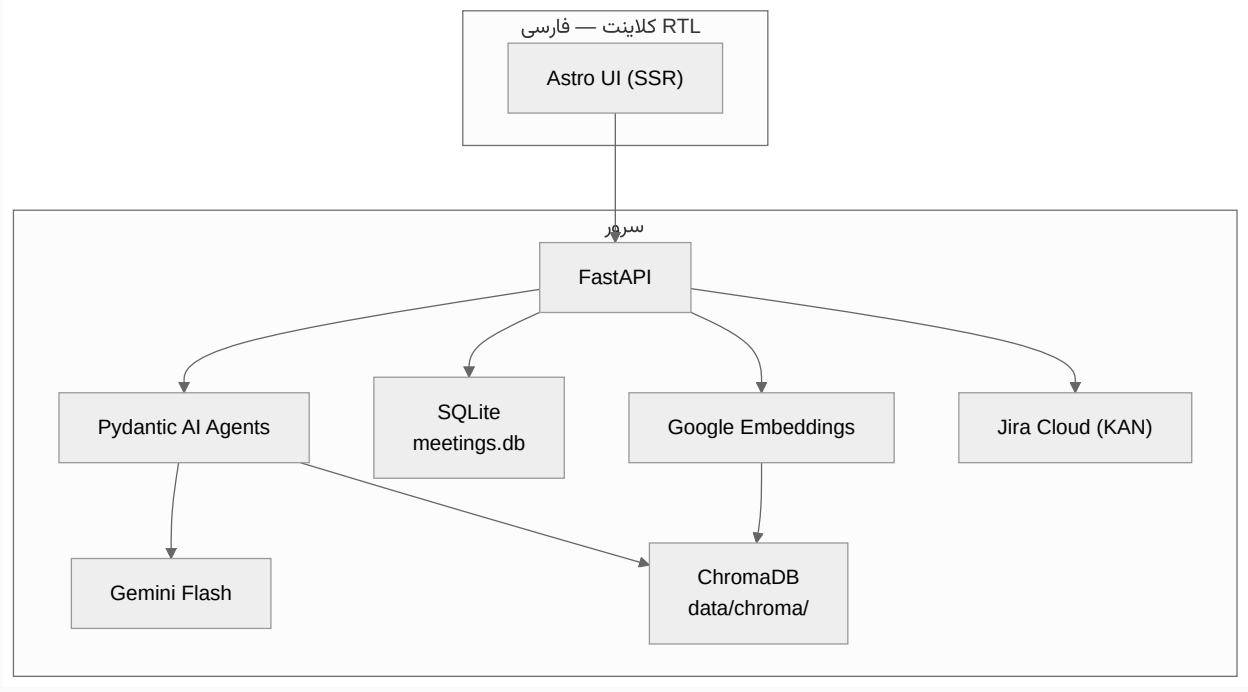
The MVP focuses on the "build and deliver" phase, but the design anticipates the full operational lifecycle of an AI product — continuous data collection, prompt and

model evaluation, deployment, and monitoring. These concerns are defined in the roadmap (document 3) rather than implemented in the MVP — an honest, realistic stance for a thesis deliverable.

> **Important note on MLOps in this project:** We do not fine-tune models; we use a hosted LLM. Therefore the operational concern is not model training but **ensuring prompt and output quality over time** — exactly where an AI product quietly degrades.

5. System Architecture

The system is a small set of separated services: an Astro frontend (server-side rendered) talking to a FastAPI backend; the backend orchestrates Pydantic AI agents on Gemini, stores meetings in SQLite, keeps embedding vectors in ChromaDB, and integrates with Jira Cloud.



Full request/response flows, database schemas, API surface, and UI are documented in detail in Document 2 — Technical Implementation.

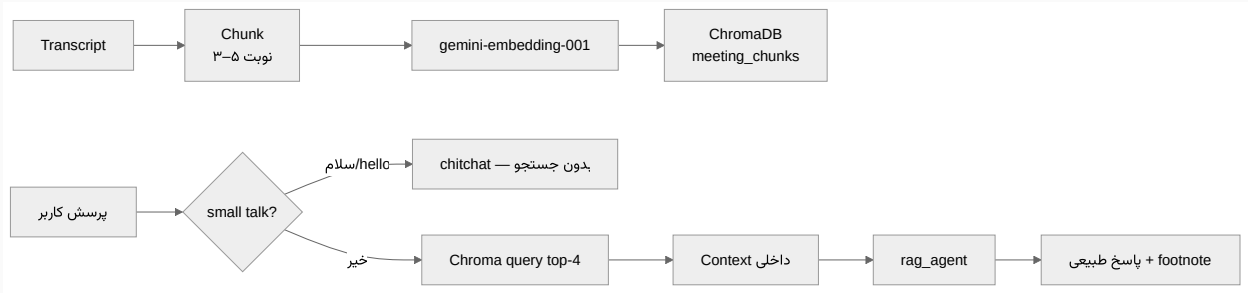
6. Agentic Design

Each AI responsibility is encapsulated in a small, single-purpose agent with a typed output contract. This keeps prompts focused, makes outputs testable, and lets the system grow by *adding agents* rather than enlarging one monolithic prompt.

Agent	Responsibility	Status
`analysis_agent`	Turn transcript into structured `MeetingAnalysis` (summary, points, decisions, tasks).	Implemented
`rag_agent`	Answer questions with retrieved context, attribution, without dumping raw transcript.	Implemented
`facilitation_agent`	Coaching suggestions for better meetings, from analysis + participation stats.	Implemented
`alignment_agent`	Compare decisions against uploaded SOW/contract (in-scope vs out-of-scope).	Planned
`sentiment_agent`	Per-speaker tone analysis on transcript text, with ethical/RBAC guards.	Planned

7. Retrieval-Augmented Generation (RAG)

On meeting ingest, the transcript is parsed into speech turns, grouped into small chunks (3–5 turns), embedded with `gemini-embedding-001`, and indexed in ChromaDB under that meeting's ID. A question is answered by retrieving the most relevant chunks and letting `rag_agent` compose a natural, grounded response.



Aspect	MVP (today)	Future
Search scope	Single meeting	Multi-meeting archive + project filter
Vector store	**ChromaDB** (embedded, `data/chroma/`)	Chroma Cloud / pgvector / Vertex
Chunking	Turn-based	Semantic with overlap

8. Technology Stack

Layer	Choice	Why
Frontend	Astro (SSR) + lightweight CSS, RTL	Meeting pages with server-side render; no heavy SPA framework needed.
Backend	FastAPI + Python 3.11+	Async, typed, excellent glue for AI services.
Agent framework	Pydantic AI	Typed outputs, native Gemini integration, minimal dependency surface.
LLM	`google:gemini-3.5-flash`	Fast, multimodal (handles audio transcription), cost-effective.

Meeting storage	SQLite (`data/meetings.db`)	Durable persistence without ops for MVP; path to PostgreSQL migration.
Vector store	ChromaDB (embedded)	No separate server; fast HNSW similarity search.
Embedding	`gemini-embedding-001`	Same provider as LLM; high-quality multilingual vectors.
Issue tracking	Jira Cloud (project `KAN`)	Industry-standard target for action items.

9. Data & Feature Engineering

The MVP ships with **four** synthetic Persian meetings (standup, planning, sprint review, and client kickoff) so the product is demoable and testable without real recordings. From each transcript, the system extracts a small set of lightweight features that feed retrieval and attribution:

Feature	Source	Use
`speaker`	Transcript parse	RAG citation and assignee hints
`timestamp`	Parse from `[HH:MM]`	Temporal ordering of chunks
`turn_count`	Chunk metadata	Context window sizing
`embedding`	Google API	Similarity search

This approach is deliberately simple: instead of a heavy feature pipeline, a few high-impact features were chosen for retrieval quality and answer readability — appropriate for an MVP.

10. Evaluation & Success Criteria

The MVP is considered successful when the following are demonstrably true:

- Three synthetic Persian meetings are processed end-to-end.
- Summary and tasks are produced at acceptable quality.
- At least three RAG questions receive natural answers with source footnotes.
- Audio transcription (upload or record) flows into the same analysis pipeline.
- At least one Jira issue is created.
- The UI is fully Persian and right-to-left.
- The automated test suite is green — **150 passing unit tests** (13 additional tests skip without optional/live credentials) plus a **13-test live API and integration suite**.

Hypothetical A/B Test (for periodic reporting)

A hypothetical A/B evaluation frames the value question for academic reporting:

Version	Change	Hypothesis
A (control)	Summary + tasks only, no RAG in UI	Baseline
B (treatment)	RAG tab enabled	Faster discovery of past decisions and higher Jira task conversion

Metrics: time to answer · citation accuracy · task-to-Jira issue conversion rate.

11. User Interface

Persian RTL interface — technical details in Document 2.

Home — ingest, archive, and sample meetings

Meeting — summary, tasks, RAG, Jira

12. What Comes Next

> The product roadmap (sprints 2–6+), advanced differentiating capabilities — facilitator guide, SOW/contract alignment, sentiment analysis, multi-meeting organizational memory, and the MLOps quality cycle — are described in Document 3 — Roadmap & Future Work. Engineering details of the current system are in Document 2 — Technical Implementation.